CUHK(SZ)-CSC3100 Midterm Exam

Sample

March 14, 2024

This paper is not the real exam paper from previous years and does not represent the actual examination question type and content.

Note:

1. No notes or calculators are allowed in the exam.
2. This exam paper has p pages, in double-sided printing.
3. Answer all questions within x minutes in an answer book.
4. Write your name and student ID on both the exam paper and answer book.

1 Multiple-choice Questions

Choose ONE solution that best suits each question. (2 points each)

D

1. (2 points) Given an array $A = [-2, 5, 9, 15, 68, 69]$. Which of the following algorithm runs fastest to sort A in ascending order?
   - A. Quick Sort
   - B. Merge Sort
   - C. Selection Sort
   - D. Insertion Sort $O(n)$

D

2. (2 points) Given an int array with size n, each number from $[0, U]$. How many times of comparison are needed to sort this array using **Counting Sort**?
   - A. O(n)
   - B. O(U)
   - C. O($n$log(n)
   - D. 0

D

3. (2 points) Given a stack, the push-order is $1, 2, 3, 4, 5$. Which of the following is **NOT** a possible pop-order?
   - A. $1, 2, 3, 4, 5$
   - B. $2, 3, 1, 5, 4$
   - C. $1, 5, 4, 3, 2$
   - D. $5, 4, 3, 1, 2$

B

4. (2 points) Which of the following statement about stack and queue is **FALSE**?
   - A. Recursion has a stronger connection with stack, rather than queue.
   - B. In an array-based stack, we need 2 integer to store the top and the bottom of the stack.
   - C. Queue is a First-in-first-out structure.

D. Array-based queue is limited on total number of items of the queue.

5. (2 points) Which of the following statement is true?

D

A. Given the pre-order and the post-order traversal of a binary tree, we can restore the tree.

B. Given the post-order traversal of a binary search tree, we cannot restore the tree.

C. Inserting a node into a binary search tree, the worst-case time complexity is O(log$n$).

D. Deleting a node from a binary search tree, the average-case time complexity is O(log$n$).

6. (2 points) Which of the following is **NOT** an advantage of linked list?

A

A. Linked list uses less memory space comparing with array with same size and data type.

B. Linked list usually works faster than array when inserting.

C. The size of a linked list is changable.

D. A linked list can contain more than one type of data.

7. (2 points) What is the time complexity of the following code?

D

```
int func(int n) {
    int sum = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = n; j > 0; j /= 2) {
            for (int k = n; k > 0; k /= 2) {
                sum++;
            }
        }
    }
    return sum;
}
```

A. O(n^3)

B. O(n^2log$n$)

C. O($n$log(n^2))

D. O(n(log$n$)^2)

8. (2 points) What is the output of func(3) given the following code?

C

```
void func(int n) {
    if (n > 0) {
        func(n-1);
        print(n);
        func(n-1);
    }
}
```

A. 12321

B. 3231323

C. 1213121

D. runtime error

9. (2 points) In a double-linked list, you can access the head and tail in O(1) time. Then what is the time complexity to combine two double-linked lists with length n into 1 circular-linked list?

C

A. O(n)

B. O(log$n$)

C. O(1)

D. illegal operation

10. (2 points) Soft-wares on your computer may raise many tasks, but your computer cannot deal with them at the same time. Usually, the earlier a task is raised, the earlier computer deals with this task. What is the data structure used here?

A. queue

B. stack

C. binary tree

D. binary search tree

2 Short Answer Questions

1. ($5 \times 4 = 20$ points) Give concise answers to the following short questions.

   (a) Describe the advantages of an array over a list. very easy to modify specific data as location is known

   (b) Explain when will the worst case for a binary search tree occur. When

   (c) If $T_1(n) = O(n^2)$ and $T_2(n) = O(n^3)$, can we conclude that $T_1(n) = O(T_2(n))$? If yes, show your proof; if no, give a counter-example.

   (d) Denote $f(x) = n^2 \log(n)$, $g(x) = \frac{1}{4}2^n + n$. Which of the statement is right, $f(x) = \Omega(g(x))$, $g(x) = \Omega(f(x))$, or both? Give your answer and prove it. $g(x) = \Omega(f(x))$

2. ($5 + 5 + 5 = 15$ points) Queue reversing. For example, the reverse of $Q = [1, 2, 3, 4]$ is $[4, 3, 2, 1]$. In this question, you can directly use basic operation of queue (e.g. enqueue, dequeue, size, isEmpty).

   (a) Given a queue Q as input. Write a function that use a stack to reverse Q, and analyze its time complexity. Write in Java or pseudo-code.

   (b) Write a function that use another queue to reverse Q, and analyze its time complexity. Write in Java or pseudo-code.

   (c) Write a function that reverse Q without directly using any other data structure. Write in Java or pseudo-code.

```
void reverse(Queue Q){
  if(Q.isEmpty()){return;}
  a = Q.dequeue();
  reverse(Q);
  Q.enqueue(a);
}
```

3. ($5 + 10 = 15$ points) Binary Search Tree.

   (a) Given an empty binary search tree (BST), insert multiple elements in the order: 5, 6, 8, 2, 10, 11, 12, 7, 4, 1. Draw the BST after inserting.

   (b) Delete the root of the BST in (a). Draw the BST after deleting. If there're more than one possible results, draw all of them.

4. (15 points) Given an array A with size n. Your task is to implement two stacks on array A. For each stack, implement isEmpty, isFull, size, push, and pop functions. Note that "isFull" returns true iff the sum of the sizes of two stacks equals n. Write in Java or pseudo-code.

5. ($7 + 8 = 15$ points) Binary Search Tree Again.

   (a) Write a function that takes a binary tree (the root of the tree) as input, and justify if the given tree is a binary search tree. Write in Java or pseudo-code.

   (b) Write a function that takes two binary search tree as input, and justify if the two BSTs contains the same elements (i.e. values, and numbers for a value are all the same). Write in Java or pseudo-code.

node bintree

bintree.h

```
class Stack{
  int a[] = new int[n];
  int key1 = 0;
  int key2 = n-1;
  void push2(int x){a[key2--] = x;}
  int pop2(){return a[++key2];}
  isEmpty{key1=0 && key2=n-1}
  isFull key1==key2+1
}
```

```
void In-order(bintree)
  if(bintree.left!=
    Inorder(bintree.left)
    list-push(bintree.ele
  if(Inorder(bintree.right)
  list
  return;
```